



PROMPT ENGINEERING AND ARTIFICIAL INTELLIGENCE · 2º SEMESTRE

Interfaces com Gradio e Streamlit + deploy com URL pública

Aula 08/14 — Módulo 3: Interfaces, Agentes e Integração

 Prof. Jorge Luiz Gomes

 1h40min

 Gradio · ngrok · streaming

 Andaime 45%

Agenda da Aula

- 1 Por que interfaces importam — de notebook a produto utilizável 10 min
- 2 Gradio — `gr.Interface`, `gr.ChatInterface`, `gr.Blocks`. Quando usar cada um. 15 min
- 3 Streaming com `yield` — respostas token por token 10 min
- 4 Deploy com ngrok — URL pública a partir do Colab 10 min
- 5 `RunnableWithMessageHistory` — memória entre turnos na interface 15 min
- 6 🚀 Demo — RAG do grupo em app Gradio ao vivo via ngrok 15 min
- 7 🖥️ Lab — cada grupo publica sua própria URL ao vivo (45% lacunas) 25 min

🎯 OBJETIVO DA AULA

Ao final da aula, o RAG do CKP02 está acessível via URL pública — qualquer pessoa com o link consegue fazer perguntas aos documentos do domínio pelo celular ou computador, sem abrir o Colab.

Glossário

gr.Blocks

API de baixo nível do Gradio para layouts customizados — controla a posição de cada componente, ao contrário de `gr.Interface` e `gr.ChatInterface`, que já vêm com layout pronto.

ngrok

Serviço que cria um túnel entre a máquina local (ou Colab) e a internet — expõe uma porta interna como URL pública acessível de qualquer dispositivo.

Analogia: um corredor temporário que liga o notebook fechado do Colab à rua aberta da internet.

session_id

Chave usada para isolar o histórico de conversa entre usuários simultâneos da mesma aplicação — cada `session_id` tem seu próprio registro, sem misturar contexto de pessoas diferentes.

st.session_state

Dicionário do Streamlit que persiste dados entre os re-renders do script — sem ele, cada interação do usuário reiniciaria as variáveis do zero.

yield (streaming)

Palavra-chave Python que transforma uma função em generator — permite devolver a resposta token a token conforme o modelo gera, em vez de esperar o texto completo.

share=True

Parâmetro nativo do `.launch()` do Gradio que gera uma URL pública temporária via infraestrutura da própria Gradio/HuggingFace — alternativa mais simples ao ngrok explícito.

RunnableWithMessageHistory

Wrapper do `LangChain` que adiciona memória de conversa a uma chain originalmente stateless — injeta o histórico correto a cada chamada, buscando pelo `session_id`.

ChatMessageHistory

Classe do `LangChain` que armazena a lista de mensagens (humano e IA) de uma sessão específica — é o objeto que o `RunnableWithMessageHistory` lê e atualiza por `session_id`.

01

De notebook a produto

Um notebook funciona para o desenvolvedor. Uma interface funciona para qualquer pessoa. Essa diferença é a diferença entre um protótipo e um produto.

Por que interfaces importam

O RAG do CKP02 funciona — mas só funciona para quem sabe abrir um notebook Python. Uma interface muda isso:

✗ SEM INTERFACE (NOTEBOOK)

Usuário precisa abrir o Colab, rodar todas as células na ordem, editar o código para trocar a pergunta, esperar output no terminal. Barreiras: Python, Colab, API key, tempo.

✓ COM INTERFACE (GRADIO + NGROK)

Usuário recebe um link, abre no celular, digita a pergunta em português, vê a resposta com citação de página em segundos. Zero barreiras de uso. Funciona em qualquer dispositivo.

https://abc123.ngrok-free.app

DocMind — Assistente do Domínio

Olá! Sou o assistente especializado nos documentos do grupo. Faça sua pergunta.

Qual é o prazo de entrega mencionado no manual?

Conforme o manual_produto.pdf, página 8: o prazo padrão é de 30 dias úteis. Para entregas expressas, a cláusula 4.2 prevê até 7 dias...

Digite sua pergunta...

Enviar

02

Gradio

gr.Interface para funções simples · gr.ChatInterface para chatbots · gr.Blocks para layouts customizados. Streaming com yield.

Gradio — quando usar cada componente

COMPONENTE	QUANDO USAR	COMPLEXIDADE	PARA O SEMESTRE
gr.Interface	Função simples: entrada → saída. Classificação, sumarização, geração de texto único.	★ Mínima — 5 linhas	Demos rápidas, exercícios
gr.ChatInterface	Chatbots conversacionais. Histórico automático. Streaming nativo com yield.	★★ Baixa — 10–15 linhas	CKP02 deploy · CKP03
gr.Blocks	Layout customizado com múltiplos inputs/outputs. Abas, colunas, eventos complexos.	★★★ Média — 30+ linhas	Diferencial CKP03

GRADIO_INTERFACE_SIMPLES.PY

```
!pip install gradio -q

import gradio as gr

# gr.Interface — a versão mais simples
def responder(pergunta: str) -> str:
    """Qualquer função Python vira uma interface."""
    return chain_rag.invoke(pergunta)

demo = gr.Interface(
    fn=responder,
    inputs=gr.Textbox(label="Pergunta", placeholder="Digite aqui..."),
    outputs=gr.Textbox(label="Resposta"),
    title="DocMind RAG",
    description="Pergunte sobre os documentos do domínio",
)

demo.launch() # abre interface no Colab inline ou em localhost
```

gr.ChatInterface — chatbot com histórico automático

`gr.ChatInterface` gerencia o histórico de conversa automaticamente e suporta streaming com `yield`:

```
import gradio as gr

# A função de chat recebe mensagem + histórico
def chat_rag(mensagem: str, histórico: list) -> str:
    """Recebe a pergunta atual e o histórico [{role, content}]."""
    # Por enquanto RAG stateless – Slide 13 adiciona memória com RunnableWithMessageHistory
    resposta = chain_rag.invoke(mensagem)
    return resposta

# ChatInterface detecta automaticamente: função recebe (mensagem, histórico) -> str
chatbot = gr.ChatInterface(
    fn=chat_rag,
    title="DocMind – Assistente do Domínio",
    description="Pergunte qualquer coisa sobre os documentos do grupo.",
    examples=[
        "Qual é o prazo de entrega mencionado no manual?",
        "Como acionar o suporte técnico?",
        "Quais são as cláusulas de garantia?",
    ],
    chatbot=gr.Chatbot(height=400),
    type="messages", # formato OpenAI: [{role, content}]
)
chatbot.launch()
```

GRADIO_CHATINTERFACE.PY

💡 TYPE="MESSAGES" — PADRÃO MODERNO

O parâmetro `type="messages"` usa o formato OpenAI com lista de dicionários `[{"role": "user", "content": "..."}]` — o mesmo formato do `RunnableWithMessageHistory`. Facilita a integração com a memória do LangChain.

Streaming com yield — respostas token a token

Sem streaming, o usuário espera a resposta completa aparecer de uma vez. Com yield, cada token aparece conforme é gerado — experiência muito mais responsiva:

❌ SEM STREAMING

SEM_STREAMING.PY

```
def chat(msg, hist):
    # Espera TUDO antes de retornar
    resp = chain_rag.invoke(msg)
    return resp # 5-30 segundos

# Usuário: 🕒 (espera 15s) → resposta aparece
```

✅ COM STREAMING (YIELD)

COM_STREAMING.PY

```
def chat_stream(msg, hist):
    parcial = ""
    # .stream() em vez de .invoke()
    for chunk in chain_rag.stream(msg):
        parcial += chunk
        yield parcial # envia a cada token

# Usuário: "Conf" → "Confor" → "Conforme..."
```

CHATINTERFACE_STREAMING.PY

```
def chat_rag_stream(mensagem: str, histórico: list):
    """Gerador: yield parcial acumulado a cada chunk."""
    parcial = ""
    for chunk in chain_rag.stream(mensagem):
        parcial += chunk # acumula — usuário vê o texto crescer
        yield parcial

# ChatInterface detecta yield automaticamente e ativa streaming
chatbot = gr.ChatInterface(fn=chat_rag_stream, type="messages",
                           title="DocMind RAG — Streaming")

chatbot.launch()
```

03

Deploy com ngrok

URL pública temporária a partir do Colab. Qualquer pessoa com o link acessa a aplicação — sem servidor, sem cloud, sem configuração extra.

ngrok — URL pública em 3 linhas

ngrok cria um túnel entre a máquina do Colab e a internet — o Gradio roda no Colab, mas a URL pública funciona de qualquer dispositivo:

- 1 **Criar conta no ngrok** — ngrok.com → plano gratuito → copiar o authtoken
- 2 **Salvar o authtoken no Colab Secrets** — ícone  no painel lateral → nome: NGROK_TOKEN
- 3 **Instalar e autenticar** — `!pip install pyngrok -q` e autenticar com o token
- 4 **Lançar o Gradio com share=True** — o Gradio já tem suporte nativo ao ngrok via parâmetro

```
DEPLOY_NGROK.PY

!pip install gradio pyngrok -q

from pyngrok import ngrok
from google.colab import userdata
import gradio as gr

# Autenticar o ngrok (gratuito – token no Colab Secrets)
ngrok.set_auth_token(userdata.get("NGROK_TOKEN"))

# Opção A – share=True (mais simples, usa infra do Gradio HuggingFace)
chatbot.launch(share=True)
# → Imprime: "Running on public URL: https://abc123.gradio.live"

# Opção B – ngrok explícito (mais controle, URL estável por sessão)
porta = 7860
tunel = ngrok.connect(porta)
print(f"URL pública: {tunel.public_url}")
chatbot.launch(server_port=porta, server_name="0.0.0.0")
# → URL válida enquanto a sessão do Colab estiver ativa
```

⚠ BOAS PRÁTICAS DE SEGURANÇA

Nunca exiba a API key na interface. Carregue sempre via `userdata.get("OLLAMA_API_KEY")` no código — jamais como variável visível ao usuário. A URL ngrok é pública: qualquer pessoa com o link pode usar a aplicação — compartilhe com consciência.

04

Memória entre turnos na interface

RunnableWithMessageHistory mantém o histórico de conversa persistido por `session_id` — cada usuário tem seu próprio contexto sem interferir nos outros.

RunnableWithMessageHistory — RAG com memória multi-sessão

A chain RAG da Aula 06 é stateless. Para que o chatbot lembre da conversa anterior (sem precisar repetir contexto), usamos

`RunnableWithMessageHistory`:

```
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_community.chat_message_histories import ChatMessageHistory

# Store em memória — uma entrada por session_id
store: dict = {}

def obter_histórico(session_id: str) -> ChatMessageHistory:
    """Retorna (ou cria) o histórico do usuário pelo session_id."""
    if session_id not in store:
        store[session_id] = ChatMessageHistory()
    return store[session_id]

# Prompt RAG com histórico (adiciona {chat_history} ao prompt)
PROMPT_RAG_HIST = """<persona>Assistente do domínio com memória de conversa.</persona>
<histórico>{chat_history}</histórico>
<contexto>{contexto}</contexto>
<instrucoes>Use o contexto para responder. Cite fontes. Se a pergunta se refere ao histórico, use-o.</instrucoes>
<pergunta>{pergunta}</pergunta>"""

chain_com_hist = (
    {
        "contexto": retriever | RunnableLambda(formatar_contexto),
        "pergunta": RunnablePassthrough(),
        "chat_history": RunnableLambda(lambda _: ""), # preenchido pelo wrapper
    }
    | ChatPromptTemplate.from_template(PROMPT_RAG_HIST)
    | llm | StrOutputParser()
)

chain_rag_memoria = RunnableWithMessageHistory(
    chain_com_hist,
    obter_histórico,
    input_messages_key="pergunta",
    history_messages_key="chat_history",
)
```

RAG_COM_MEMORIA.PY

Gradio + RunnableWithMessageHistory — chatbot completo

CHATBOT_RAG_MEMORIA_GRADIO.PY

```
import uuid, gradio as gr

def chat_com_memoria(mensagem: str, histórico: list, session_id: str):
    """Streaming com memória por session_id."""
    parcial = ""
    for chunk in chain_rag_memoria.stream(
        {"pergunta": mensagem},
        config={"configurable": {"session_id": session_id}}, # chave do histórico
    ):
        parcial += chunk
        yield parcial

with gr.Blocks(title="DocMind RAG") as demo:
    # session_id gerado automaticamente por sessão de browser
    session_id = gr.State(lambda: str(uuid.uuid4()))

    gr.Markdown("## 📄 DocMind — Assistente do Domínio")
    chatbot_ui = gr.ChatInterface(
        fn=chat_com_memoria,
        type="messages",
        additional_inputs=[session_id], # passa o session_id como input extra
        examples=["Qual é o prazo de garantia?", "Como acionar o suporte?"],
    )

demo.launch(share=True) # → URL pública gerada automaticamente
```

👤 GR.STATE — ESTADO POR SESSÃO DE BROWSER

`gr.State(lambda: str(uuid.uuid4()))` gera um UUID único para cada nova aba ou sessão de browser que abre a aplicação. O `RunnableWithMessageHistory` usa esse UUID como chave para manter históricos separados por usuário — múltiplos usuários simultâneos sem interferência.

Streamlit — alternativa com st.session_state

Streamlit tem uma sintaxe mais próxima de scripts Python tradicionais. `st.session_state` persiste dados entre re-renders:

STREAMLIT_RAG.PY

```
!pip install streamlit -q # rodar como app separado: !streamlit run app.py &

import streamlit as st

st.title("DocMind – Assistente do Domínio")

# Inicializar histórico no session_state (persiste entre re-renders)
if "mensagens" not in st.session_state:
    st.session_state.mensagens = []

# Exibir mensagens anteriores
for msg in st.session_state.mensagens:
    with st.chat_message(msg["role"]):
        st.write(msg["content"])

# Input do usuário
if pergunta := st.chat_input("Faça sua pergunta..."):
    st.session_state.mensagens.append({"role": "user", "content": pergunta})
    with st.chat_message("user"):
        st.write(pergunta)
    with st.chat_message("assistant"):
        resposta = st.write_stream(chain_rag.stream(pergunta)) # streaming nativo
    st.session_state.mensagens.append({"role": "assistant", "content": resposta})
```

Gradio vs. Streamlit — quando usar cada um

CRITÉRIO	GRADIO	STREAMLIT
Instalação e setup	Roda inline no Colab com <code>demo.launch()</code>	Precisa rodar como processo separado (<code>streamlit run app.py</code>)
ChatInterface	Nativo — <code>gr.ChatInterface</code> em 5 linhas	Manual com <code>st.chat_message</code> + <code>session_state</code>
File upload	<code>gr.File()</code> — retorna caminho do arquivo	<code>st.file_uploader()</code> — retorna bytes
Deploy	<code>share=True</code> → URL instantânea	Streamlit Cloud ou ngrok manual
Layouts complexos	<code>gr.Blocks</code> para layouts customizados	Colunas, tabs e sidebar nativos
Para o semestre	✅ CKP02, CKP03 — padrão da disciplina	Alternativa válida no diferencial

🔗 RECOMENDAÇÃO PARA O CKP02 E CKP03

Use **Gradio** — menos fricção no Colab, deploy com `share=True` em 1 linha, `gr.ChatInterface` com streaming pronto. O Streamlit é ótimo para dashboards com múltiplos painéis — se o grupo quiser um layout mais elaborado, vai para `gr.Blocks` antes de migrar para Streamlit.

📖 FUNDAMENTAÇÃO — POR QUE O STREAMLIT NÃO ESCALA COMO O GRADIO

Polzer (*RAG with Python Cookbook*, O'Reilly, 2026) explica o mecanismo por trás da linha "Deploy" desta tabela: o Streamlit reexecuta o script inteiro do zero a cada interação do usuário — isso simplifica o gerenciamento de estado, mas cria sobrecarga de CPU por usuário, e o autor situa o ponto de saturação em torno de 50 usuários simultâneos. Por isso a recomendação do livro é usar Streamlit para ferramentas internas e provas de conceito, migrando para FastAPI com React quando o público crescer para milhares de usuários ou exigir autenticação refinada. Para os projetos desta disciplina — poucos usuários simultâneos, demonstração em sala — essa diferença de escala é irrelevante; o critério que importa é fricção de setup, e é aí que o Gradio ganha.

06

Lab — Notebook Aluno

45% de lacunas — cada grupo publica sua própria URL ao vivo. URL compartilhada no grupo da turma ao final.

```
!pip install gradio langchain-community langchain-ollama chromadb pymupdf -q

import gradio as gr, uuid, os
from google.colab import userdata
from langchain_community.chat_message_histories import ChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.runnables import RunnablePassthrough, RunnableLambda

os.environ["OLLAMA_HOST"] = "https://ollama.com"
os.environ["OLLAMA_API_KEY"] = userdata.get("OLLAMA_API_KEY")

# Reutilizar db do CKP02 (já indexado - só recarregar)
embeddings = OllamaEmbeddings(model="nomic-embed-text")
db = Chroma(persist_directory="/content/ckp02", embedding_function=embeddings)

# 🚩 LACUNA 1: crie o retriever com k=3
retriever = db.as_retriever(search_kwargs={"k": ( )})

# 🚩 LACUNA 2: monte a chain RAG com memória
store = {}
def obter_hist(sid):
    if sid not in store: store[sid] = ChatMessageHistory()
    return store[sid]

chain_base = (
    {"contexto": retriever | RunnableLambda(formatar_contexto),
     "pergunta": ( ),
     "chat_history": RunnableLambda(lambda _: "")}
    | prompt_com_hist | llm | StrOutputParser()
)
chain_mem = RunnableWithMessageHistory(
    chain_base, obter_hist,
    input_messages_key=( ),
    history_messages_key=( ),
)

# 🚩 LACUNA 3: função de chat com streaming
def chat(msg, hist, sid):
    parcial = ""
    for chunk in chain_mem.stream({"pergunta": ( ),
                                   config={"configurable": {"session_id": ( )}}):
        parcial += chunk
    yield parcial

# 🚩 LACUNA 4: monte a interface e publique com share=True
with gr.Blocks() as demo:
    sid = gr.State(lambda: str(uuid.uuid4()))
    gr.Markdown("## 📁 DocMind do Grupo - [Nome do Domínio]")
    gr.ChatInterface(fn=( ), type="messages", additional_inputs=[ ( ) ])
demo.launch(share=( )) # True para URL pública
```

Boas práticas de deploy e segurança

PRÁTICA	COMO IMPLEMENTAR	POR QUÊ
Nunca expor API keys	Sempre via <code>userdata.get()</code> — jamais como texto na interface	A URL ngrok é pública — qualquer usuário pode inspecionar o código-fonte da página
Timeout handling	Envolver <code>chain.stream()</code> em try/except e fazer yield de mensagem de erro	O Ollama Cloud pode ter latência variável — o usuário não deve ver um app travado
Limitar o input	<code>gr.Textbox(max_lines=5)</code> ou truncar a string antes de invocar	Inputs muito longos consomem tokens e podem causar erros no modelo
Resetar sessão	Adicionar botão que limpa o <code>store[session_id]</code>	Memória crescente degrada a qualidade (context rot) — permitir ao usuário recomeçar
Mensagem de carregamento	Primeira yield com "⌚ Buscando nos documentos..."	RAG tem latência de retrieval — o usuário precisa saber que o app está trabalhando

Síntese da Aula 08



Interface = produto. `gr.ChatInterface` transforma o pipeline RAG em app utilizável por qualquer pessoa em 15 linhas. `share=True` gera a URL.



Streaming com yield muda a experiência: o usuário vê tokens chegando em tempo real em vez de esperar o bloco completo por 15 segundos.



RunnableWithMessageHistory adiciona memória por `session_id` — múltiplos usuários simultâneos, cada um com seu próprio histórico isolado.



SOON

Aula 09: o chatbot vira um agente. Em vez de apenas buscar no RAG, o agente vai decidir quando usar o RAG, quando pesquisar na web e quando usar uma calculadora — autonomamente.

Perguntas frequentes

PERGUNTA	RESPOSTA
"A URL ngrok expira? Quanto tempo dura?"	No plano gratuito, a URL expira quando a sessão do Colab termina (timeout de ~90 min de inatividade ou 12h máximas). Para demos ao vivo em sala, é suficiente. Para demonstrações mais longas, usar ngrok pago ou hospedar no HuggingFace Spaces (gratuito, URL permanente).
"Posso usar gr.Blocks para adicionar upload de PDF na interface?"	Sim — <code>gr.File(file_types=[".pdf"])</code> retorna o caminho temporário do arquivo enviado. Na função de upload: carregar com PyMuPDF, dividir, embedar e adicionar ao ChromaDB com <code>db.add_documents()</code> . O desafio do lab pede exatamente isso.
"HuggingFace Spaces é uma alternativa ao ngrok?"	Sim, e melhor para URLs permanentes. O Spaces suporta Gradio nativamente — você faz push de um repositório com <code>app.py</code> e <code>requirements.txt</code> e a URL fica no ar permanentemente. O desafio: as API keys precisam ser configuradas nas variáveis de ambiente do Space (não via Colab Secrets).
"O histórico cresce infinitamente? Precisa limitar?"	Sim — sem limite, o histórico sofre context rot. Na produção, use <code>ConversationTokenBufferMemory</code> (aprendida na Aula 02) para limitar por tokens. Para o lab desta aula, o histórico ilimitado é aceitável em demo.

Python novo desta aula

CONCEITOS_PYTHON_AULA08_2SEM.PY

```
# 1. yield – transforma função em gerador (streaming)
def gerar_números():
    for i in range(5):
        yield i      # pausa aqui, retorna i, retoma na próxima iteração
list(gerar_números()) # → [0, 1, 2, 3, 4]

# 2. Walrus operator := – atribuição dentro de if
if pergunta := st.chat_input("..."): # atribui E testa ao mesmo tempo
    print(pergunta) # só entra se pergunta não for None/vazio

# 3. lambda sem argumento – gr.State usa para valor inicial por sessão
uuid_factory = lambda: str(uuid.uuid4()) # chamada a cada nova sessão
estado       = gr.State(uuid_factory)    # sem () – passa a função, não o resultado

# 4. += em string (acumular streaming)
parcial = ""
for chunk in ["Conf", "orme", " a p", "ágina"]:
    parcial += chunk # acumula: "Conf" → "Conforme" → "Conforme a p" → ...
    yield parcial   # envia o estado acumulado a cada passo

# 5. Dict aninhado como config do LangChain
config = {"configurable": {"session_id": meu_session_id}}
chain.stream(input, config=config) # config é kw-only, sempre nomeado
```

Interfaces de IA em produção

HUBS DE MODELOS ABERTOS

Milhares de demos com Gradio

Comunidades de machine learning mantêm repositórios públicos com milhares de demonstrações de modelos, boa parte construída com Gradio — o mesmo `gr.Interface`/`gr.ChatInterface` que você usou hoje, publicado em escala.

ASSISTENTES CONVERSACIONAIS

Streaming como padrão de mercado

Assistentes de IA conversacional amplamente usados hoje entregam a resposta token a token — o mesmo mecanismo de `yield` implementado nesta aula. A diferença entre uma resposta que só aparece após 15 segundos e uma que começa a aparecer em milissegundos é inteiramente de streaming, não do modelo em si.

STARTUPS BR

Gradio como MVP

Startups brasileiras de LegalTech, HealthTech e EdTech usam Gradio para lançar MVPs em semanas. O RAG + Gradio + ngrok é exatamente o stack de MVP mais rápido disponível em 2026.

O que vem na Aula 09 — Agentes ReAct e tools

A DIFERENÇA ENTRE CHAIN E AGENTE

Chain (o que você tem hoje):

Pergunta → retriever → prompt → LLM → resposta

Fluxo fixo, pré-determinado.

Agente (o que vem na Aula 09):

Pergunta → LLM *decide* qual tool usar → executa → observa

→ LLM decide próximo passo → ... → resposta

Fluxo dinâmico, guiado pelo raciocínio do modelo.

O QUE VOCÊ VAI IMPLEMENTAR

Agente ReAct com 3 tools:

- **RAG tool** — busca nos documentos do grupo (CKP02)
- **DuckDuckGo tool** — pesquisa na web quando o doc não tem a resposta
- **Calculadora** — para perguntas numéricas

O agente decide qual usar para cada pergunta — autonomamente.

 **Antes da Aula 09:** mantenha o notebook desta aula salvo com a interface funcionando. A Aula 09 vai trocar a chain pelo agente — mas a interface Gradio permanece a mesma.

05

Demo ao vivo

RAG do grupo em app Gradio. 15 linhas adicionais além do pipeline da Aula 06.

URL pública via share=True. Acesso pelo celular em sala.

Demo Prática ao Vivo — RAG do grupo em 15 linhas extras

O professor abre o notebook do CKP02, adiciona as células abaixo e a interface vai ao ar:

DEMO_DEPLOY_A0_VIVO.PY

```
!pip install gradio -q # já deve estar instalado

import gradio as gr, uuid

# — Memória por sessão —————
store = {}
def obter_hist(sid):
    if sid not in store: store[sid] = ChatMessageHistory()
    return store[sid]

# — Chat com streaming + memória —————
def chat(msg, hist, sid):
    parcial = ""
    for chunk in chain_rag_memoria.stream(
        {"pergunta": msg},
        config={"configurable": {"session_id": sid}},
    ):
        parcial += chunk
    yield parcial

# — Interface —————
with gr.Blocks(title="DocMind RAG") as demo:
    sid = gr.State(lambda: str(uuid.uuid4()))
    gr.Markdown("# 📄 DocMind – Assistente do Domínio\nPergunte sobre os documentos do grupo.")
    gr.ChatInterface(fn=chat, type="messages", additional_inputs=[sid])

demo.launch(share=True)

# → "Running on public URL: https://abc123.gradio.live"
# → professor projeta o QR Code – alunos acessam pelo celular
```

Publicar a primeira URL ao vivo do grupo ★★

Grupo 3-4 · 25 minutos · Google Colab

1. **Complete as 4 lacunas** — retriever, chain com memória (input/history keys), função de chat com streaming e lançamento com `share=True`.
2. **Personalize a interface** — título do grupo, 3 exemplos de perguntas reais do domínio como botões de atalho.
3. **Publique e compartilhe** — copie a URL gerada e poste no grupo da turma. Acesse a URL de outro grupo e faça 2 perguntas.
4. **Documente:** a interface lembrou o contexto da primeira pergunta na segunda? Teste perguntas de acompanhamento tipo "e sobre isso que você disse..."

🎯 Gabarito das lacunas

Lacuna 1: `"k": 3`

Lacuna 2: `RunnablePassthrough()` para `"pergunta"`; `input_messages_key="pergunta"`; `history_messages_key="chat_history"`

Lacuna 3: `msg` para `"pergunta"`; `sid` para `"session_id"` no config

Lacuna 4: `chat` para `fn`; `sid` para `additional_inputs`; `True` para `share`

🚀 **Desafio:** adicionar um `gr.Blocks` com aba para upload de novo PDF e reindexação ao vivo — sem reiniciar o app.

Referências

DOCS **Gradio** — ChatInterface, Blocks, streaming e deploy. gradio.app/docs/gradio/chatinterface

DOCS **LangChain** — RunnableWithMessageHistory para múltiplas sessões. python.langchain.com/docs/how_to/message_history

DOCS **ngrok** — Túnel HTTP gratuito para desenvolvimento e demos. ngrok.com/docs

DOCS **Streamlit** — st.chat_message, st.session_state, file_uploader. docs.streamlit.io/develop/api-reference/chat

LIVRO **Russell, S.; Norvig, P.** — Inteligência Artificial. 3ª ed. Pearson, 2016. Cap. 2 — Agentes inteligentes: a fundamentação teórica da transição de pipeline para agente que acontece nas próximas aulas.

Primeira URL pública ao vivo

De notebook a produto. O RAG do grupo agora tem uma URL e qualquer pessoa pode perguntar sobre os documentos do domínio pelo celular. Próximo: transformar isso em agente.

PRÓXIMA AULA — AULA 09

Agentes de IA — ReAct, tools e function calling

O chatbot passa a decidir qual ferramenta usar. RAG, web search e calculadora — autonomamente.

Salvar

Notebook desta aula com interface funcionando. A Aula 09 troca só a chain pelo agente.